TCS302 - DATA STRUCTURES WITH C

Merge Sort

Algorithm, Implementation and Complexity

Unit IV - Lecture 5

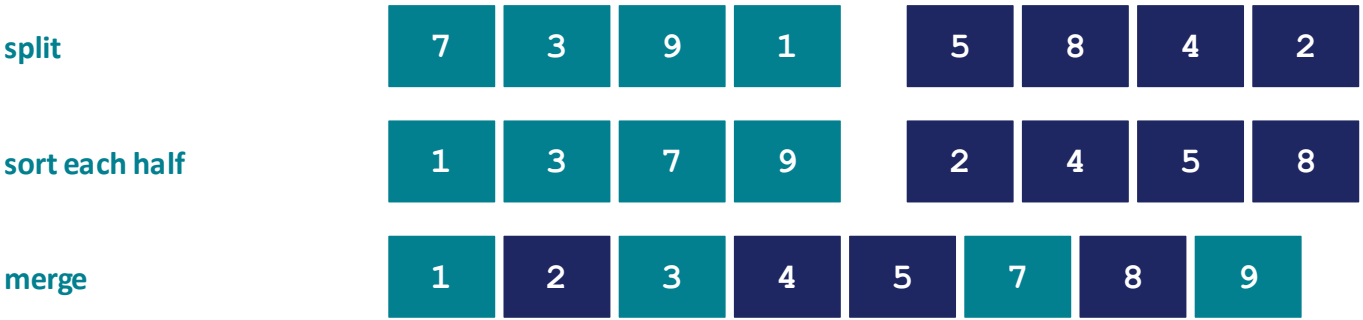
Idea split in halves, sort each half, then merge the two sorted halves

Dry run merge traced on 3 7 9 and 2 5 8, whole sort on 7 3 9 1 5 8 4 2

Complexity O(n log n) in every case, at the price of an extra array, verified

The idea: split, sort each half, merge

Split the array in the middle. Sort each half by the same method. Merge the two sorted halves into one sorted array. A single element is already sorted, which is where the splitting stops.



Splitting is free. All the work is in the merge, which builds one sorted list by always taking the smaller front value of the two halves.

The algorithm, step by step

1. mergeSort(a, lo, hi): if lo >= hi stop. One element is already sorted.
2. mid = lo + (hi - lo) / 2. Sort lo..mid, then sort mid+1..hi.
3. merge(a, lo, mid, hi): i walks the left half, j walks the right half, k fills tmp.
4. Compare a[i] with a[j]. Copy the smaller one to tmp (the LEFT one on a tie) and advance that index.
5. When one half runs out, copy the rest of the other half, then copy tmp back into a[lo..hi].

Merge invariant
At every step tmp[0..k-1] holds the k smallest values of the two halves, in sorted order. Each comparison adds the next smallest value.

Merge sort in C

```
void merge(int a[], int lo, int mid, int hi)
{
    int tmp[MAX];               /* helper */
    int i = lo, j = mid + 1, k = 0;

    while (i <= mid && j <= hi)
    {
        if (a[i] <= a[j]) { tmp[k] = a[i]; i++; }
        else              { tmp[k] = a[j]; j++; }
        k++;
    }
    while (i <= mid) { tmp[k] = a[i]; i++; k++; }
    while (j <= hi)  { tmp[k] = a[j]; j++; k++; }

    for (k = 0; k < hi - lo + 1; k++)
        a[lo + k] = tmp[k];
}
```

```
void mergeSort(int a[], int lo, int hi)
{
    if (lo < hi)
    {
        int mid = floor( (lo + hi) / 2);
        mergeSort(a, lo, mid);
        mergeSort(a, mid + 1, hi);
        merge(a, lo, mid, hi);
    }
}
```

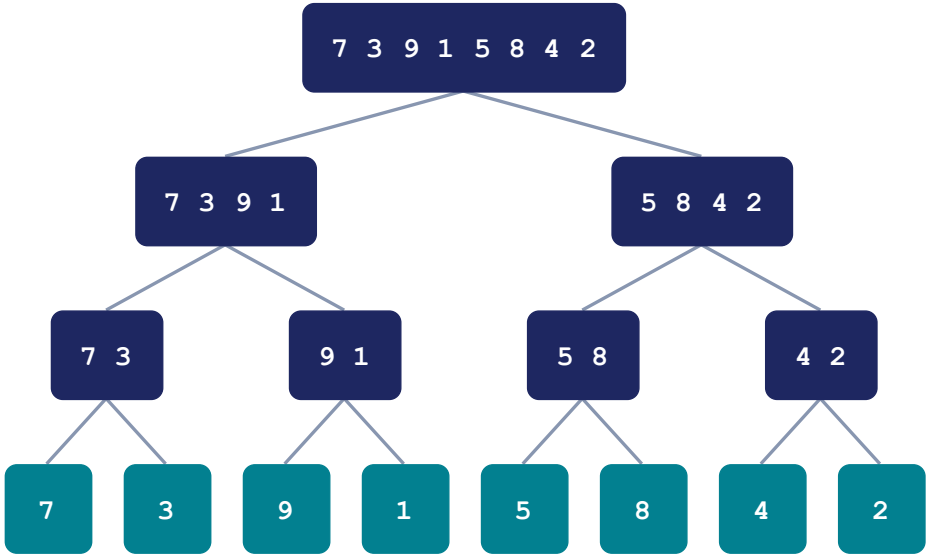
- On a tie (<=) the LEFT value goes first: this keeps merge sort stable.
- tmp holds the merged result until it is copied back into a[lo..hi].
- MAX must be at least n. The extra array is the price of merge sort.

One merge: 3 7 9 with 2 5 8 (verified)

Step	Compare	Action	tmp after the step
start	left 3 7 9, right 2 5 8	i at 3, j at 2	(empty)
1	3 vs 2	take 2 (right)	2
2	3 vs 5	take 3 (left)	2 3
3	7 vs 5	take 5 (right)	2 3 5
4	7 vs 8	take 7 (left)	2 3 5 7
5	9 vs 8	take 8 (right)	2 3 5 7 8
6	right half is empty	copy the leftover 9	2 3 5 7 8 9

5 comparisons for 6 values. The leftover 9 is copied without any comparison, so a merge of n1 and n2 values needs at most n1 + n2 - 1 comparisons.

Split down, merge up: 7 3 9 1 5 8 4 2



Splitting goes down; the merges happen on the way back up.

Comparisons while merging

Level 1 (four merges): 1+1+1+1 = 4

Level 2: 3 + 2 = 5

Level 3 (final merge): 7

Total: 16

Result: 1 2 3 4 5 7 8 9. Splits are free; comparisons happen only while merging.

Best, worst and average case, measured (n = 8)

Case	Input	Comparisons
Best	`1 2 3 4 5 6 7 8  (already sorted)`	12
Best	`8 7 6 5 4 3 2 1  (reversed)`	12
Typical	`5 2 8 1 9 3 7 4`	17
Average	`mean over all 40320 orderings`	15.73
Worst	`1 3 2 7 4 6 5 8`	17

- Over all 40320 orderings of eight values the comparisons range only from 12 to 17: there is no bad input.
- Sorted input still costs 12 comparisons, not 7: merge sort is not adaptive. Quick sort needed 28 on the same input.
- Best, average and worst case are all O(n log n).

Growth: the same n log n pattern for every input

n	merge, sorted	merge, random	quick, sorted	quick, random
100	356	550	4950	667
200	812	1291	19900	1517
400	1824	2977	79800	3749
800	4048	6762	319600	7871

Merge sort
Doubling n costs about 2.2 to 2.35 times as much, for sorted and random input alike: the n log n pattern.

Quick sort (last-element pivot)
Quadruples on sorted input. Same random arrays, one fixed array per size (seed 12345).

The price: an extra array

n	extra array used by merge sort	merge sort max depth	quick sort max depth, sorted input
100	100 values	8	100
400	400 values	10	400
1000	1000 values	11	1000

The cost
The final merge needs a helper array as big as the data itself. Quick sort needs no second array.

The benefit
Splitting is always in the middle, so the recursion stays about log2 n deep: 11 for n = 1000, whatever the input.

Properties of merge sort

Property	Answer	Evidence
Stable	Yes	A tie takes the left value. Marks Asha 60, Bina 60, Charu 75, Dev 80 keep their order
In-place	No	Needs an extra array of n values (1000 for n = 1000)
Adaptive	No	Sorted input, n = 8: 12 comparisons (insertion sort: 7)
Time: best / average / worst	O(n log n) in all three	12 to 17 comparisons for n = 8 over all orderings

Mixed input, n = 8 (5 2 8 1 9 3 7 4). Comparisons: merge sort 17, quick sort 16, insertion sort 18, bubble sort with the flag 25. On 1000 pseudo-random values merge sort needs 8729 comparisons and quick sort 11251.

Split, sort, merge

- Split in halves, sort each half, merge. The merge always takes the smaller front value; a tie takes the left one.
- Merging 3 7 9 with 2 5 8 takes 5 comparisons; the whole sort of 7 3 9 1 5 8 4 2 takes 16 in three levels of merging.
- O(n log n) in every case (12 to 17 comparisons for n = 8). Stable, but not in-place and not adaptive: it needs an extra array of n values.
- On 1000 sorted values: merge sort 5044 comparisons, quick sort with a last-element pivot 499500.

EXIT TICKET Merge the sorted lists 2 6 9 and 1 5 7 8. Show the tmp array after each step and count the comparisons.